

We now turn to the second task, namely, how to use the recurrence formulas (6.5.6) and (6.5.7) to get the Bessel functions $J_n(x)$ and $Y_n(x)$ for $n \geq 2$. The latter of these is straightforward, since its upward recurrence is always stable:

```
bessel.h  Doub Bessjy::yn(const Int n, const Doub x)
Returns the Bessel function  $Y_n(x)$  for any positive  $x$  and integer  $n \geq 0$ .
{
    Int j;
    Doub by,bym,byp,tox;
    if (n==0) return y0(x);
    if (n==1) return y1(x);
    tox=2.0/x;
    by=y1(x);           Starting values for the recurrence.
    bym=y0(x);
    for (j=1;j<n;j++) {   Recurrence (6.5.7).
        byp=j*tox*by-bym;
        bym=by;
        by=byp;
    }
    return by;
}
```

The cost of this algorithm is the calls to y_1 and y_0 (which generate a call to each of j_1 and j_0), plus $O(n)$ operations in the recurrence.

For $J_n(x)$, things are a bit more complicated. We can start the recurrence upward on n from J_0 and J_1 , but it will remain stable only while n does not exceed x . This is, however, just fine for calls with large x and small n , a case that occurs frequently in practice.

The harder case to provide for is that with $x < n$. The best thing to do here is to use Miller's algorithm (see discussion preceding equation 5.4.16), applying the recurrence *downward* from some arbitrary starting value and making use of the upward-unstable nature of the recurrence to put us *onto* the correct solution. When we finally arrive at J_0 or J_1 we are able to normalize the solution with the sum (5.4.16) accumulated along the way.

The only subtlety is in deciding at how large an n we need start the downward recurrence so as to obtain a desired accuracy by the time we reach the n that we really want. If you play with the asymptotic forms (6.5.3) and (6.5.5), you should be able to convince yourself that the answer is to start larger than the desired n by an additive amount of order $[\text{constant} \times n]^{1/2}$, where the square root of the constant is, very roughly, the number of significant figures of accuracy.

The above considerations lead to the following function.

```
bessel.h  Doub Bessjy::jn(const Int n, const Doub x)
Returns the Bessel function  $J_n(x)$  for any real  $x$  and integer  $n \geq 0$ .
{
    const Doub ACC=160.0;           ACC determines accuracy.
    const Int IEXP=numeric_limits<Doub>::max_exponent/2;
    Bool jsum;
    Int j,k,m;
    Doub ax,bj,bjm,bjp,dum,sum,tox,ans;
    if (n==0) return j0(x);
    if (n==1) return j1(x);
    ax=abs(x);
    if (ax*ax <= 8.0*numeric_limits<Doub>::min()) return 0.0;
    else if (ax > Doub(n)) {       Upwards recurrence from  $J_0$  and  $J_1$ .
        tox=2.0/ax;
```

```

    bjm=j0(ax);
    bj=j1(ax);
    for (j=1;j<n;j++) {
        bjp=j*tox*bj-bjm;
        bjm=bj;
        bj=bjp;
    }
    ans=bj;
} else {
    tox=2.0/ax;
    m=2*((n+Int(sqrt(ACC*n)))/2);
    jsum=false;
    bjp=ans=sum=0.0;
    bj=1.0;
    for (j=m;j>0;j--) {
        bjm=j*tox*bj-bjp;
        bjp=bj;
        bj=bjm;
        dum=frexp(bj,&k);
        if (k > IEXP) {
            bj=ldexp(bj,-IEXP);
            bjp=ldexp(bjp,-IEXP);
            ans=ldexp(ans,-IEXP);
            sum=ldexp(sum,-IEXP);
        }
        if (jsum) sum += bj;
        jsum=!jsum;
        if (j == n) ans=bjp;
    }
    sum=2.0*sum-bj;
    ans /= sum;
}
return x < 0.0 && (n & 1) ? -ans : ans;
}

```

Downward recurrence from an even m here computed.

$jsum$ will alternate between false and true; when it is true, we accumulate in sum the even terms in (5.4.16).
The downward recurrence.

Renormalize to prevent overflows.

Accumulate the sum.
Change false to true or vice versa.
Save the unnormalized answer.

Compute (5.4.16) and use it to normalize the answer.

The function `ldexp`, used above, is a standard C and C++ library function for scaling the binary exponent of a number.

6.5.1 Modified Bessel Functions of Integer Order

The modified Bessel functions $I_n(x)$ and $K_n(x)$ are equivalent to the usual Bessel functions J_n and Y_n evaluated for purely imaginary arguments. In detail, the relationship is

$$\begin{aligned}
 I_n(x) &= (-i)^n J_n(ix) \\
 K_n(x) &= \frac{\pi}{2} i^{n+1} [J_n(ix) + i Y_n(ix)]
 \end{aligned}
 \tag{6.5.13}$$

The particular choice of prefactor and of the linear combination of J_n and Y_n to form K_n are simply choices that make the functions real-valued for real arguments x .

For small arguments $x \ll n$, both $I_n(x)$ and $K_n(x)$ become, asymptotically, simple powers of their arguments

$$\begin{aligned}
 I_n(x) &\approx \frac{1}{n!} \left(\frac{x}{2}\right)^n & n \geq 0 \\
 K_0(x) &\approx -\ln(x) \\
 K_n(x) &\approx \frac{(n-1)!}{2} \left(\frac{x}{2}\right)^{-n} & n > 0
 \end{aligned}
 \tag{6.5.14}$$